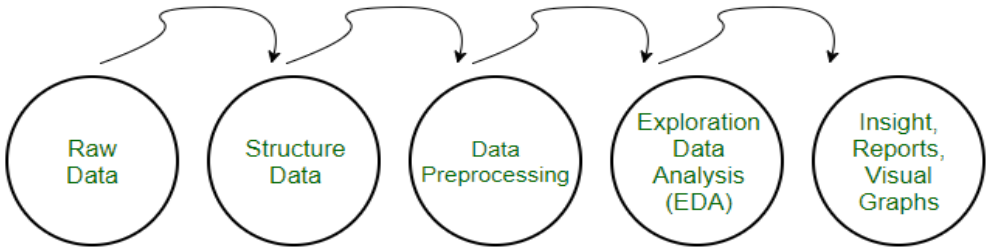
Data Analytics Using Python

Complete End to End Guide

Python has emerged as a preferred tool for data analysis due to its simplicity, versatility, and many open-source libraries. With its intuitive syntax and large online community, Python enables both beginners and experts to perform complex data analysis tasks efficiently.

Data analytics sections:

1. *Data Pre-processing*
2. *Data Analysis*
3. *Data Visualization*



1. Data Pre-processing

Data Processing means preparation of data for analysis in the working environment. Data pre-processing is an important step in the data science transforming raw data into a clean structured format for analysis. It involves tasks like handling missing values, normalizing data and encoding variables.

Without doing manual work for this repetitive work python ***allows*** us to automate data cleaning using mainly 2 libraries, which are

- Numpy as np
- Pandas as pd

We mostly use Pandas

Elements of Data Pre-processing:

- Finding & Replacing Missing Values
- Finding & Replacing Duplicate Values

1st Chapter Reference:

- https://www.geeksforgeeks.org/machine-learning/data-preprocessing-machine-learning-python/

2. Data Analysis

Data Analysis is the technique of collecting, transforming and organizing data to make future predictions and informed data-driven decisions. These analysis done by using majorly 2 libraries called,

- Numpy for numerical analysis
- Pandas for tabular data analysis

We mostly use Pandas

Numpy operations:

- Addition
- Subtraction
- Multiplication
- Division

Analyzing Data Using Pandas

Pandas Is used for relational or labeled data and provides various data structures for manipulating such data and time series. Pandas generally provide two data structures for manipulating data, They are:

- Series as ser
- Data Frame as dr

We mostly use Data Frame

Data Frame:

Data Frame consists of three principal components i.e data, rows and columns.

Operations of Data Frame:

- Filtering
- Sorting
- Grouping or Groupby
- Aggregation

- Concatenating
- Merging
- Joining

2nd Chapter References:

- https://www.geeksforgeeks.org/data-analysis/data-analysis-with-python/
- https://www.geeksforgeeks.org/data-analysis/exploratory-data-analysis-in-python/

3. Data Visualization

Data visualization is more than just creating pretty charts; it's about turning complex data into actionable insights. For Data Analysts, Business Analysts, and Data Scientists, visualizations are indispensable tools for communicating findings, spotting trends, and making data-driven decisions.

Python offers a range of powerful libraries for data visualization, each with its unique features and advantages. Here's an overview of the most widely used and reliable Python libraries for creating stunning visualizations:

- Matplotlib as plt
- Seaborn as sns
- Plotly as px
- Bokeh as bkp
- Altair as alt

We mostly use Matplotlib & Seaborn

Visualizing Data using Matplotlib

- Line chart
- Bar chart
- Pie chart
- Histogram
- Scatter plot
- Box plot
- Heatmap

3rd Chapter References:

- https://scribe.rip/@nomannayeem/mastering-data-visualization-with-python-an-end-to-end-guide-84334b28087c
- https://www.geeksforgeeks.org/data-visualization/data-visualization-using-matplotlib/

Final References

- https://pyoflife.com/python-for-data-analysis-a-complete-guide-for-beginners/
- https://www.simplilearn.com/tutorials/python-tutorial/data-visualization-in-python
- https://saturncloud.io/blog/a-quick-guide-to-exploratory-data-analysis-using-jupyter-notebook/
- https://www.geeksforgeeks.org/machine-learning/data-preprocessing-machine-learning-python/
- https://www.geeksforgeeks.org/data-analysis/exploratory-data-analysis-in-python/
- https://www.geeksforgeeks.org/data-visualization/data-visualization-using-matplotlib/
- https://learnpython.com/blog/python-exploratory-data-analysis-cheat-sheet/
- https://learnpython.com/blog/python-data-cleaning/
- https://www.geeksforgeeks.org/data-analysis/data-analysis-with-python/
- https://www.geeksforgeeks.org/data-analysis/types-of-statistical-data/
- https://scribe.rip/@nomannayeem/mastering-data-visualization-with-python-an-end-to-end-guide-84334b28087c
- https://github.com/NikamAshutosh/Exploratory-Data-Analysis-EDA-Using-Python/blob/main/Exploratory%20Data%20Analysis%20Using%20Python/Exploratory_Data_Analysis%20(Sales%20Data).ipynb

In order to use those libraries we need to import them into our environment, just like the below code

```
>>> import pandas as pd
>>> import numpy as np
```

Data processing section consists of the following steps:

Step 1: Importing Dataset

Dataset can be in different formats like excel (.xlxs), comma separated values (.csv). Mostly csv format is preferred.

```
>>> import pandas as pd
>>> import numpy as np
>>> df = pd.read_csv (“diabetic-dataset.csv”)
```

If you get errors or struggling to import dataset, even if you copied the file path as below check the following examples;

```
>>> df = pd.read_csv ("C:\Users\lette\Data Visualization\diabetes.csv")
```

it will show syntax error. To solve this error follow any of the options below

```
#Option 1 here we place r at starting of the file path
>>> df = pd.read_csv (r"C:\Users\lette\Data Visualization\diabetes.csv")

#Option 2  here we use double \\
>>> df = pd.read_csv("C:\\Users\\lette\\Data Visualization\\diabetes.csv")

#Option 3 here we use /
>>> df = pd.read_csv ("C:/Users/lette/Data Visualization/diabetes.csv")
```

Let’s start by getting an overview of our dataset:

```
df.head()
```

The df.head() function prints the first 5 rows of data. You can see the row number (starting from zero), the date (in yyyy-mm-dd format), and the observation of the number of sunspots for the month.

Output:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

1. Check the data info

df.info()

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

As we can see from the above info that the our dataset has 9 columns and each columns has 768 values. There is no Null values in the dataset.

2. Check the null values

df.isnull()

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...
763	False	False	False	False	False	False	False	False	False
764	False	False	False	False	False	False	False	False	False
765	False	False	False	False	False	False	False	False	False
766	False	False	False	False	False	False	False	False	False
767	False	False	False	False	False	False	False	False	False

768 rows × 9 columns

3 Checking the sum of Null Values

df.isnull() .sum()

```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

4 Checking duplicates

df.drop_duplicates()

df.drop_duplicates() .sum() ----------->- for knowing total duplicates

Step 3: Statistical Analysis

In statistical analysis we use df.describe() which will give a descriptive overview of the dataset.

OUTPUT:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

The above table shows the count, mean, standard deviation, min, 25%, 50%, 75% and max values for each column. When we carefully observe the table we will find that Insulin, Pregnancies, BMI, BloodPressure columns has outliers.